

Simularea deplasării într-un sistem planetar

Conceptul

Proiectul urmărește crearea unui cadru care simulează călătoria inter-planetară într-un sistem solar ficțional. Sistemul solar conține un număr mare de planete care dețin o varietate de caracteristici și particularități (culoarea, distanța orbitei, viteza de orbitare, numărul de sateliți, viteza de revoluție în jurul axei proprii, etc.) și dă posibilitatea utilizatorului să le exploreze suprafața. În implementarea acestora se urmărește și demonstrarea utilității mai multor concepte din grafica pe calculator, precum:

- Manipularea nivelului de detaliu al unui render (LoD)
- Simularea mișcării unui observator (atât deplasare cât și mișcarea camerei)
- Utilizarea stivei de matrici pentru a controla poziția planetelor
- Implementarea unui sistem de iluminare complex

Scenele generate în cadrul acestui proiect pot fi împărțite în două categorii: scene de suprafață și scene spațiale.

Scenele de suprafață urmăresc reprezentarea unui teren populat de vegetație care simulează prezența pe un corp ceresc. Utilizatorul se poate mișca liber în acest spațiu, schimbându-și chiar și altitudinea pentru a putea decola de pe suprafața planetei. Numărul de poligoane utilizat pentru alcătuirea suprafeței este redus când altitudinea crește, pentru a conserva resurse computaționale.

În urma atingerii altitudinii de 20 000 de metri, scena este schimbată, iar utilizatorul este trimis în spațiul inter-planetar. Aceste tipuri de scene reprezintă mai multe planete, orbitând în jurul unui astru central care le și iluminează. Mișcarea liberă a observatorului este implementată și aici, și poate fi folosită pentru a explora spațiul generat sau pentru a ateriza pe o anumită planetă (după ce intră în atmosfera sa).

Implementarea grafică

Separarea dintre implementările celor două scene este asigurată de implementarea unei structurii de tip OOP (Object Oriented Programming), unde clasa `ModelManager` proiectează scena de suprafață, iar clasa `SolarSystem` se ocupă de spațiul inter-planetar. Tranziția dintre

aceste două scene se efectuează în funcția principală de rendering, prin intermediul parametrului boolean `renderMode`.

- Când parametrul este setat la 0 (generând suprafața), iar altitudinea observatorului crește peste 20 000 de metri, `renderMode` devine 1 și se trece la scena spațială.
- Când parametrul este 1 și este detectată o coliziune (distanța dintre utilizator și centrul unei planete este mai mică sau egală ca raza acesteia + o constantă), `renderMode` devine 0 și se trece la scena de suprafață.

Implementarea suprafeței necesită utilizarea mai multor obiecte (pentru teren, pentru cer, etc.), comunicarea acestora cu funcțiile din main fiind asigurată de clasa `ModelManager`. De aici putem accesa clasele pentru `Terrain` și `Skybox`, care se ocupă cu desenarea propriu zisă a obiectelor.

Clasa `Terrain` se folosește de împărțirea în patch-uri pentru a genera o suprafață infinită. Bineînțeles, doar o parte din acest obiect infinit este desenat și prezentat utilizatorului, iar de decuparea corectă a acestei părți se ocupă parametrii `lod` (`Level of Detail`). În funcție de distanța de rendering și de punctul către care se uită camera este generată o bucată din suprafață, aceasta fiind mai apoi trimisă către shader unde este augmentată prin introducerea perlin noise-ului și a efectelor vizuale. La final este adăugată și vegetația, aceasta fiind reprezentată de copaci compuși din forme simple.

Generarea cerului este comparativ mai simplă, nefiind nevoie de decupare sau modificare a nivelului de detaliu. În clasa `Skybox` este generat un VAO care să deseneze cerul din mai multe triunghiuri, fiecare dintre acestea având o culoare ce diferă subtil. Aceste diferențe dau un aspect radial cerului și creează impresia unor raze solare.

Generarea spațiului inter-planetar se realizează prin clasa `SolarSystem`. Această clasă utilizează mai multe funcții auxiliare (`Draw Planet`, `Draw Asteroid Ring`) pentru a genera un număr mare de planete diferite. Acestea sunt create cu ajutorul clasei `Sphere`, mai apoi urmând să fie transmise către Shaderul Vertex împreună cu o matrice de model care calculează poziția acesteia în scenă. Funcția `Draw Planet` primește mai multe argumente care influențează poziția eventuală a planetei, precum:

- `parentMatrix`, pentru a sincroniza poziția față de corpul orbitat
- `orbitSpeed`, pentru viteza orbitării
- `orbitAxis`, pentru axa orbitei
- `orbitTranslation`, distanța față de corpul orbitat
- `spinSpeed`, viteza revoluției în jurul axei proprii
- `spinAxis`, axa proprie

- scaleVector, pentru redimensionare
- planetColor, culoarea de bază a planetei
- MyShader, shaderul care va prelua renderingul planetei
- registerCollision, pentru a detecta observatorul
- emissiveStrength, pentru iluminarea corectă

Generarea centurii de asteroizi se realizează similar celei pentru sateliți. Un număr de sfere mici este generat la poziții, înălțimi, înclinații, și dimensiuni cu variațiuni subtile cu ajutorul unei funcții de zgomot pentru a crea aspectul haotic al unei centuri de asteroizi.

Pentru a gestiona numărul mare de shadere utilizat de diversele componente ale scenelor (terenul, cerul, planetele, sateliții, etc.), se utilizează în proiect și un ShaderManager. Aceasta este o clasă care inițializează și apoi facilitează accesul prin furnizarea pointerilor către mai multe Shadere specializate.

O ultimă componentă tehnică a proiectului ar fi mișcarea observatorului și a camerei. Input-urile de la tastatură sunt detectate și procesate de către InputManager care transmite informația către un obiect de tip Camera. Aceasta se mișcă în plan folosind tastele WASD (WS pentru față-spate și AD pentru stânga dreapta) și efectuează rotații de tip pitch (prin săgețile sus-jos) sau yaw (prin săgețile stânga-dreapta). Mișcarea camerei se poate realiza și prin intermediul mouse-ului.

Contribuțiile Membrilor

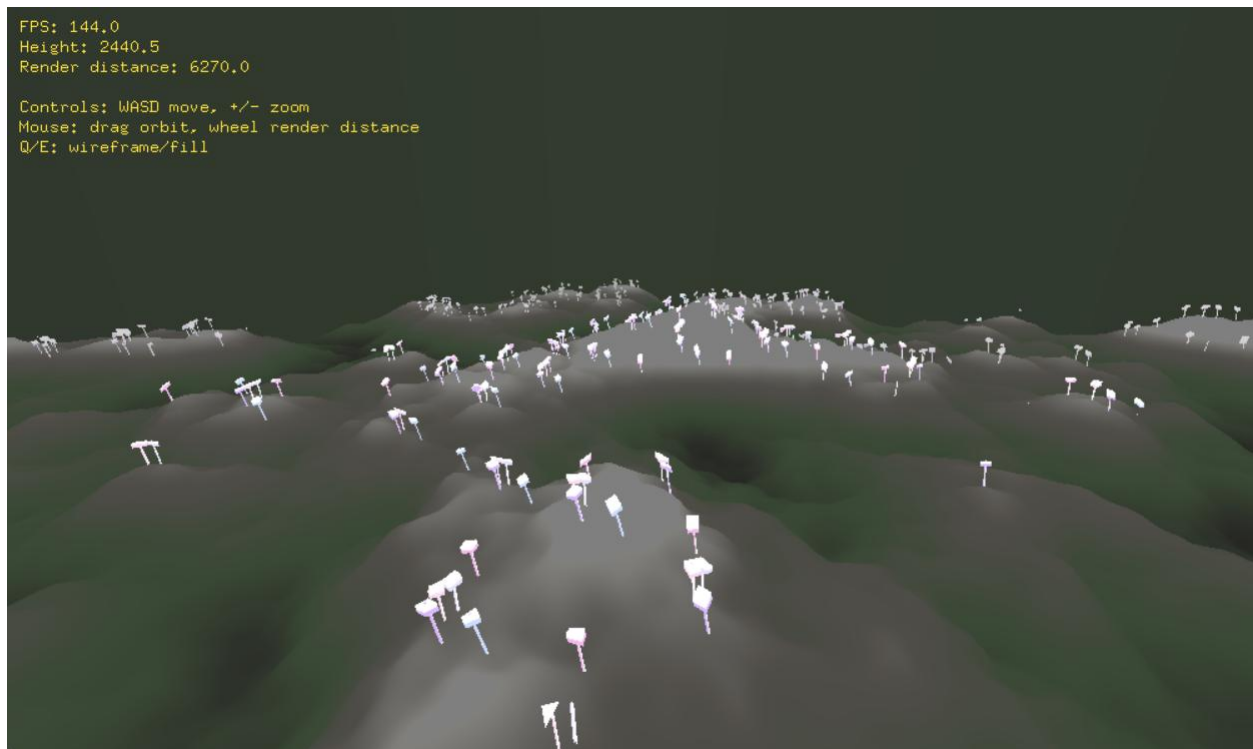
Membru	Contribuții principale
Antonio Soare	Generarea scenei de suprafață(teren + cer). Implementarea structurii de OOP a proiectului.
Mihnea Andreescu	Implementarea funcțiilor de desenare pentru asteroizi. Desenarea sistemului solar și utilizarea sistemului de iluminare.
Ștefan Militaru	Implementarea funcțiilor de desenare pentru planete. Implementarea tranzițiilor dintre suprafață și spațiu.

Elemente Originale

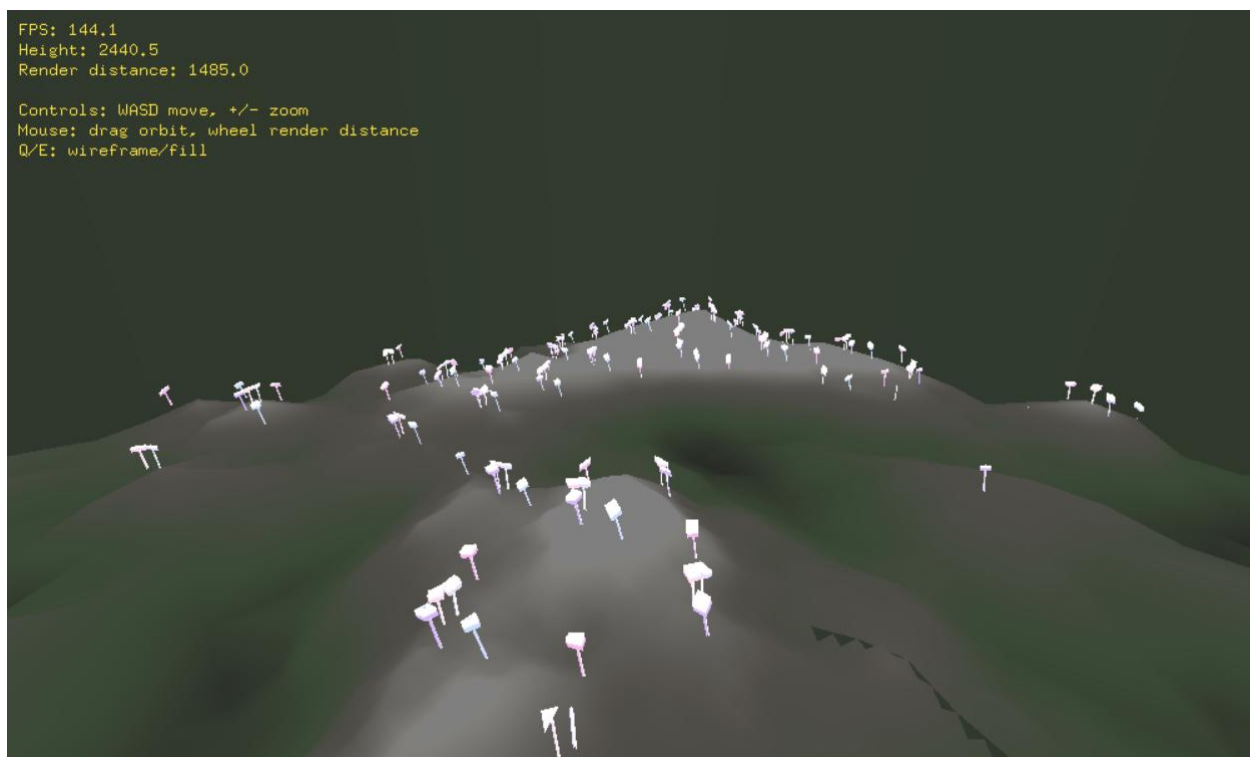
Pentru a reprezenta câteva părți mai complexe din conceptul proiectului a fost necesară utilizarea unor aspecte originale sau care nu au fost prezentate la laborator. Printre acestea se numără:

- Împărțirea suprafeței în patch-uri pentru a genera un teren infinit ce nu dă impresia că se repetă.
- Prevenirea discontinuităților între patchuri prin implementarea perlin noise-ului direct în shader.
- Utilizarea noise-ului pentru a genera vegetație într-un mod realist (apariția vegetației e legată de valoarea zgomotului în patch-ul curent).
- Adăugarea efectelor vizuale (ceață, materiale diferite, iluminare) atât în scenele de suprafață (pentru a da terenului o apariție naturală) cât și în scenele spațiale (pentru a simula efectul razelor solare asupra planetelor).
- Schimbarea dinamică a nivelului de detaliu pentru a conserva resurse computaționale. Acesta scade în funcție de distanța scenei față de observator, dar poate fi modificat manual cu ajutorul roțiței de la mouse.
- Structurarea codului ca un program OOP, prin utilizarea de clase pentru a desena obiectele necesare fiecărei scene. Cele două scene sunt fiecare implementate de câte o clasă, care la rândul ei are nevoie de alte clase pentru fiecare element din cadrul său.

Rezultatul



Scena de suprafață cu un LoD mare



Acceași scenă, însă cu un LoD mic



Scena spațială